Flag Detection & Classification Pipeline

This document details the step-by-step image processing methodology used to automatically detect and classify flags (specifically distinguishing between Indonesia and Poland) using OpenCV.

1. Reading and Resizing the Image

```
image = cv2.imread(image_path)
image = cv2.resize(image, (400, 300))
```

- The image is loaded into memory from the specified file path.
- It is immediately resized to a standardized dimension of 400x300 pixels. This ensures uniform processing and reduces computational overhead.

2. Blurring to Remove Noise

```
blurred = cv2.GaussianBlur(image, (5, 5), 0)
```

- A Gaussian Blur is applied to the image to smooth out minor irregularities, such as fabric wrinkles, texture, or digital artifacts.
- This is a crucial pre-processing step to guarantee clean and consistent color segmentation.

3. Converting to HSV Color Space

```
hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)
```

- The color representation is transformed from the standard BGR format to HSV (Hue, Saturation, Value).
- HSV is significantly more robust for color-based detection because it isolates the actual color pigment (Hue) from its purity (Saturation) and brightness (Value).

4. Creating Color Masks

Specific color masks are generated to isolate the red and white elements of the flag.

```
# Red Mask
red_lower1 = np.array([0, 70, 50])
red_upper1 = np.array([10, 255, 255])
red_lower2 = np.array([160, 70, 50])
red_upper2 = np.array([180, 255, 255])
red_mask = cv2.inRange(hsv, red_lower1, red_upper1) | cv2.inRange(hsv,
red_lower2, red_upper2)
```

- Because the red hue wraps around the boundaries of the HSV scale (appearing near both 0 and 180), two distinct ranges are defined. A bitwise OR operation merges them to capture all red pixels comprehensively.

```
# White Mask
white_lower = np.array([0, 0, 200])
white_upper = np.array([180, 50, 255])
white_mask = cv2.inRange(hsv, white_lower, white_upper)
```

- White is characterized by low saturation and high brightness (Value). This configuration reliably isolates those bright, low-saturation regions.

5. Identifying the Flag Region

```
combined_mask = cv2.bitwise_or(red_mask, white_mask)
contours, _ = cv2.findContours(combined_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

if not contours:
    return "Unable to detect flag region"

largest_contour = max(contours, key=cv2.contourArea)
x, y, w, h = cv2.boundingRect(largest_contour)
flag_roi = image[y:y+h, x:x+w]
```

- The red and white masks are combined, and the algorithm identifies continuous external contours within this merged mask.
- Operating under the assumption that the largest contiguous region of red and white corresponds to the flag, it extracts the bounding box of this largest contour to form a cropped Region of Interest (ROI).

6. Resizing and Segmenting the Flag

```
flag_roi = cv2.resize(flag_roi, (200, 100))
flag_hsv = cv2.cvtColor(flag_roi, cv2.COLOR_BGR2HSV)
top_half = flag_hsv[:50, :]
bottom_half = flag_hsv[50:, :]
```

- The extracted flag ROI is standardized to exactly 200x100 pixels.
- It is then spatially segmented into a top half (rows 0-50) and a bottom half (rows 50-100) to prepare for structural color analysis.

7. Measuring Color Ratios

Helper functions are utilized to calculate the exact proportion of red and white pixels present in both halves.

```
top_red = red_ratio(top_half)
top_white = white_ratio(top_half)
bottom_red = red_ratio(bottom_half)
bottom_white = white_ratio(bottom_half)
```

8. Final Classification Logic

The system evaluates the spatial distribution of the identified colors to make a final classification.

```
if top_red > bottom_red and bottom_white > top_white:
    return "Indonesia"
elif top_white > bottom_white and bottom_red > top_red:
    return "Poland"
else:
    return "Unable to classify confidently"
```

- If the top segment is predominantly red and the bottom segment is predominantly white, the script classifies the flag as **Indonesia**.
- Conversely, if the top segment is white and the bottom segment is red, it outputs **Poland**.
- If neither condition is definitively met, it falls back to an unconfident classification message.